

## 3.4.3 Operators and Operands: Arithmetic and Assignment

آپرینڈ (Operand) وہ ویلیو ہے جس پر عمل ہوتا ہے اور آپریٹر (Operator) وہ علامت ہے جو عمل کرتی ہے۔

CS XI

Chapter 3: Programming Fundamentals

Lecture 10 of 24

★ Golden

3.4.3 Operators and Operands: Arithmetic and Assignment ★ Golden

LEARN IT

An **operand** is the value or variable being worked on. An **operator** is the symbol that performs the action. Together they form an **expression**, which Python evaluates to a single value. In `10 + 20`: `10` is an operand, `+` is the operator, `20` is an operand, and the result is `30`.

1. Arithmetic operators do maths:

| Operator        | Purpose                                 | Example             | Result |
|-----------------|-----------------------------------------|---------------------|--------|
| <code>+</code>  | Addition                                | <code>5 + 3</code>  | 8      |
| <code>-</code>  | Subtraction                             | <code>10 - 4</code> | 6      |
| <code>*</code>  | Multiplication                          | <code>4 * 2</code>  | 8      |
| <code>/</code>  | Division (result is always a float)     | <code>9 / 2</code>  | 4.5    |
| <code>//</code> | Floor division (drops the decimal part) | <code>9 // 2</code> | 4      |
| <code>%</code>  | Modulus (remainder)                     | <code>9 % 2</code>  | 1      |
| <code>**</code> | Exponentiation (power)                  | <code>2 ** 3</code> | 8      |

```
print(5 + 3)
print(10 - 4)
print(4 * 2)
print(9 / 2)
print(9 // 2)
print(9 % 2)
print(2 ** 3)
print(8 / 2)
```

```
OUTPUT
8
6
8
4.5
4
1
8
4.0
```

Notice the last line: `8 / 2` gives `4.0`, not `4`, because `/` always gives a float.

**Order of operations (precedence).** Python follows the same order as maths (like BODMAS): brackets first, then `**`, then `*` / `/` / `%` (left to right), then `+` / `-` (left to right). `**` is the exception: it works right to left.

```
print(2 + 3 + 4)
print((2 + 3) + 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(-9 // 2)
print(17 % 5, 17 // 5)
```

```
OUTPUT
14
20
512
3
-5
2 3
```

`2 ** 3 ** 2 = 2 ** 9 = 512` (right to left). `-9 // 2` is `-5` because floor division always rounds **down** to the smaller whole number (`-4.5 → -5`).

2. **Assignment operators** store a value in a variable, often updating it. The short forms are called **compound assignment**.

| Operator                                                                | Example             | Equivalent to                      |
|-------------------------------------------------------------------------|---------------------|------------------------------------|
| <code>=</code>                                                          | <code>x = 5</code>  | Assign 5 to x                      |
| <code>+=</code>                                                         | <code>x += 2</code> | <code>x = x + 2</code>             |
| <code>-=</code>                                                         | <code>x -= 1</code> | <code>x = x - 1</code>             |
| <code>*=</code>                                                         | <code>x *= 3</code> | <code>x = x * 3</code>             |
| <code>/=</code> , <code>//=</code> , <code>%=</code> , <code>**=</code> | <code>x /= 2</code> | <code>x = x / 2</code> (and so on) |

WORKED EXAMPLE

Tracing assignment operators. Follow the value of `x` line by line:

```
x = 5
x += 2      # x = 5 + 2
print(x)
x -= 1      # x = 7 - 1
print(x)
x *= 3      # x = 6 * 3
print(x)
x /= 2      # x = 18 / 2
print(x)
```

```
OUTPUT
7
6
18
9.0
```

Using `%` and `//` to split digits. `% 10` gives the last digit; `// 10` removes it.

```
n = 357
last = n % 10      # remainder after dividing by 10
rest = n // 10     # drop the last digit
print("Last digit:", last)
print("Remaining:", rest)
```

```
OUTPUT
Last digit: 7
Remaining: 35
```

`357 ÷ 10 = 35 remainder 7`. So `357 // 10 = 35` and `357 % 10 = 7`.

REMEMBER IT

**Three kinds of division:** `/` = exact answer (float), `//` = quotient only ("floor"), `%` = remainder only. A number is **even** when `n % 2 == 0`.

COMMON MISTAKE

`^` is not power in Python — it is bitwise XOR (see bitwise operators). `2 ^ 3` gives 1, not 8. For power, always use `**`.

آرڈو میں سمجھیں

اپریٹنڈ (Operand) وہ ویلیو ہے جس پر عمل ہوتا ہے اور آپریٹر (Operator) وہ علامت ہے جو عمل کرتی ہے۔ / ہمیشہ اعشاریہ والا جواب دیتا ہے، // صرف حاصل تقسیم دیتا ہے، % باقی دیتا ہے اور \*\* طاقت (Power) نکالتا ہے۔ 2 + x کا مطلب ہے x + 2.

EXAM TIP

You will be asked to evaluate expressions such as `17 // 5`, `17 % 5`, `2 ** 4` or to trace compound assignment. Show each step and write the final value with its type (e.g. `9.0` is a float, so write the `.0`).

CHECK YOURSELF

1. Predict the output:

```
x = 7
y = 2
print(x // y, x % y, x / y)
print(x ** y)
```

2. Predict the output:

```
x = 10
x += 5
x /= 4
print(x)
```

Answers: 1. 3 1 3.5 then 49. 2. 3 (10 + 5 = 15; 15 / 4 = 3).

---

Next lecture: 3.4.3 Relational, Logical and Membership Operators